

BEHAVIORAL COLLAPSE IN AI-GENERATED HARNESS POPULATIONS

Anonymous authors

Paper under double-blind review

ABSTRACT

Populations of automatically generated LLM harnesses are increasingly used for routing and value prediction — applications that presuppose population members *disagree* on outcomes. We measure this premise directly and find that it fails. On BIRD text-to-SQL with a frozen target, the candidates of a standard harness-evolution loop are syntactically diverse yet behaviorally collapsed: 36 of 91 harness pairs (including the bare and ReAct baselines) return identical correctness on every task, and pairwise code similarity shows no observed rank association with outcome disagreement (Spearman $\rho = -0.01$). Execution-trace auditing explains why: mechanisms are described but absent from the control flow, feedback has no causal path to the output, or multi-turn paths are broken. A hand-written positive control shows the missing routing headroom exists on the task (7.3 pp), localizing the failure to generation; we further document a more primitive failure mode in which a deployed pipeline emits candidates byte-identical to the baseline. We then froze a confirmatory protocol: a 2×2 factorial (strategy forcing $\times$ counterfactual conformance gate) over six builders and three seeds, evaluated on 1,169 questions from databases never opened during development, under the official BIRD scorer, with instruments calibrated on positive and negative controls. Phase-I controls additionally show open-set harness-value prediction does not beat the best fixed harness, so diversity is necessary but not sufficient. Confirmatory data collection is ongoing; results will be reported under the frozen analysis plan.

1 INTRODUCTION

The fastest-growing lever on a frozen LLM is no longer weights but the *harness* — the program of prompts, tools, and control flow that wraps the model for a task. A growing line of “AI4AI” systems generates this scaffolding automatically: LLM builders propose, edit, and evolve harness code. The implicit premise is that when the generated code differs, the *behavior* differs — that a population of thirty harnesses is thirty ways of answering, not one way written thirty times. Routing and value prediction over such populations presuppose exactly this.

Phase I (discovery): the premise fails, and for identifiable reasons. On BIRD text-to-SQL (Li et al., 2023) with a frozen mid-tier target, we measure what the candidates of a standard harness-evolution loop actually *do*, not what their source files say (Figure 1). Every candidate passes a code-difference check, yet among the 12 generated candidates (66 candidate pairs) average pairwise outcome disagreement is **3.8%** and **21 pairs are behaviorally identical**; counting the shipped `react` harness and `bare` baseline as well (91 pairs), 36 pairs return identical correctness on every task, and pairwise code similarity shows no observed rank association with outcome disagreement (Spearman $\rho = -0.01$). The shipped `react` variant reproduces `bare`’s final SQL byte-for-byte on every audited task. Execution-trace auditing resolves “different code” into three failure classes: mechanisms *described* but absent from the control flow, feedback with no causal path to the output, and genuinely multi-turn paths that are broken (§3). We call this **behavioral collapse**.

Collapse is not a property of the task: four hand-written harnesses with forcibly distinct execution paths deliver 7.3 pp oracle headroom on the same target and judge — the headroom routers need is real; the generation side fails to populate it. We also document a second, more primitive failure mode in one deployed pipeline: a widely used optimize loop emitted candidates *byte-identical to the baseline source*. This is not behavioral collapse — no distinct population was generated at all

— and we report it as a *generation-null* (M0) failure, kept strictly separate from the collapse claim, which we support only on genuinely distinct-source populations.

Phase II (confirmatory): a pre-registered factorial. Phase I suggested that behavioral collapse can be repaired by generation-side interventions, but its design — two nested contrasts, three builders, unlogged early runs — could not separate protocol from builder effects, its evaluation set overlapped development items, and its judge was a conservative proxy. We therefore froze a confirmatory protocol before any confirmatory outcome was observed: a 2×2 factorial over *strategy forcing* (assigned mechanism vs. free invention) and a *mechanism-conformance gate* (counterfactual trace verification vs. neutral validity only), crossed with six builder models and three seeds, evaluated on 1,169 questions from nine databases never opened during development, scored by the official BIRD judge, with the primary contrast (forced+gated vs. free+ungated populations) analyzed as paired continuous effects with hierarchical bootstrap intervals (§4). The conformance instruments are calibrated against hand-written positive controls and known-negative impostor harnesses before any candidate is scored.

Our contributions:

1. **An outcome-matrix measurement of behavioral collapse** in AI-generated harness populations, with an execution-trace taxonomy explaining why code diversity fails to become behavioral diversity, and a clean separation between genuine collapse and generation-null failures.
2. **Calibrated generation-side instruments and a pre-registered factorial study:** a counterfactual conformance suite and self-specified behavioral contracts, both calibrated on positive and negative controls; a 2×2 design with equal raw generation budgets that isolates the combined intervention and its components on uncontaminated data under the official scorer.
3. **A boundary result for routing:** on the Phase-I populations, open-set harness-value prediction does not beat the best fixed harness, and its apparent per-task signal is consistent with task memorization; we scope this precisely to the tested predictors and splits (§6).

Beyond harnesses, the failure pattern we isolate — LLM builders producing plausible, well-documented, behaviorally inert variants — is a generation-reliability phenomenon we expect to recur wherever a model is asked to modify its own program.

2 RELATED WORK

2.1 HARNESS GENERATION FOR LLMs

A growing line of work generates task-specific scaffolding around a frozen model. TTHe evolves text-to-SQL harnesses with an LLM proposer/judge loop (Nie et al., 2026); Self-Harness (Zhang et al., 2026b) shows a model can improve its own harness; JIT-Agent (Zhang et al., 2026a) trains a model to generate a harness per task at inference time; Task-CoEvolve (Miyai et al., 2026) co-evolves tasks and scaffolds. A concurrent 2026 cluster makes harness evolution *behavior-aware from the verification side*: HarnessLens (Xu et al., 2026) allocates rollout budget using behavior-level checks during evolution; HarnessFix (Chen et al., 2026) repairs harnesses from trace-grounded diagnoses; AHE (Lin et al., 2026) verifies each proposed edit against self-declared behavioral predictions; gated semantic quality-diversity search (Luo et al., 2026) evolves populations over LLM-assigned pathology descriptors; Harness Handbook (Wang et al., 2026) attacks behavior localization and outer-loop search.

Our work is complementary but distinct in unit of analysis and question. Table 1 contrasts the two nearest systems. HarnessLens uses behavior as a *signal inside* an evolution loop to allocate compute; we ask whether the resulting *population* is behaviorally diverse as an outcome property, and what generation-side factors cause or repair its collapse. HarnessBank (Luo et al., 2026) documents search-space collapse and maintains a semantic diversity library with gated screening; our conformance gate does not score semantics at all — it verifies mechanically, via counterfactual trace probes, that a declared mechanism executes — and our design isolates the gate’s causal contribution through a factorial contrast rather than reporting end-to-end system gains.

	This work	HarnessLens	HarnessBank
Unit of study	population outcome matrix	evolution loop	diversity library
Measures population collapse	yes	no	search collapse only
Gate definition	counterfactual traces	attributable-evidence	semantic screening
Gate calibration	pos/neg controls	—	—
Causal isolation of gate	2×2 factorial	gate ablation (single factor)	no
Confirmatory split	9 untouched DBs	—	—

Table 1: Nearest concurrent systems. Behavior-aware verification inside evolution systems is concurrent independent work; our contribution is the population-level outcome measurement, instrument calibration, and the controlled generation-side contrast.

2.2 HARNESS CONDITIONAL UTILITY AND ROUTING

Several recent papers establish that harness quality is conditional on the instance: Gupta et al. (2026) and the non-monotonicity analysis of Cho (2026) document that no single harness dominates across instances. HELIX (Fan & Huang, 2026) shows a portfolio of sibling harnesses beats the best fixed one and lists a harness router as future work; STS (Zhou et al., 2026) routes over six fixed human-designed scaffolds in a closed set; GRASP (Moll et al., 2026) gates skill edits into a library at development time; Kim (2026) learn counterfactual instance-level routing over {no-search, search, abstain}; Liu et al. (2026) route models within a harness. All of these consume a population whose members *actually disagree*; Fan & Huang (2026) comes closest, evaluating a full candidate outcome matrix and measuring portfolio complementarity beyond the best fixed candidate. Our contribution is narrower and different: we measure whether *automatically generated* populations collapse as an outcome phenomenon, diagnose why at the trace level, and isolate the generation-side causes with a controlled factorial. show it fails for naively generated populations, and test whether generation-side interventions repair it.

2.3 BEHAVIORAL DIVERSITY IN LLM POPULATIONS

Diversity of LLM outputs has mostly been studied at the sample level (self-consistency, diverse decoding) or the agent-policy level. The observation that syntactically distinct programs can exhibit identical behavior echoes coverage gaps documented in program synthesis and mutation-testing practice; our contribution is to instantiate and quantify it for AI-generated harness populations, with a mechanism-level taxonomy.

3 PHASE I: MEASURING AND DIAGNOSING THE COLLAPSE (DISCOVERY)

Phase I is a discovery study: its role is to establish the phenomenon, the taxonomy, and the existence of task-side headroom, and to motivate the confirmatory design. None of its results is confirmatory: 24 of its 151 evaluation items overlapped protocol-development measurements, the human control was designed after inspecting development traces, and its judge was a conservative proxy (Appendix A). We report it with those limits stated.

3.1 HARNESSES, OUTCOMES, AND THE THREE METRICS

A **harness** H wraps a frozen target model M : it receives the natural-language task x , issues one or more calls to M , optionally executes intermediate artifacts (e.g. SQL), and returns a final answer scored programmatically. With H_0 the bare single-call baseline and $Y \in \{0, 1\}$ correctness, the **harness marginal value** is $\Delta(x, H, M) = Y(x, H, M) - Y(x, H_0, M) \in \{-1, 0, +1\}$ (+1 repair, -1 harness-induced harm).

We propose that harness-generation and harness-routing pipelines report three precondition metrics on a held-out set, all computable from the outcome matrix any evaluation already produces: (i) **pairwise outcome disagreement** — over all harness pairs, the fraction of tasks where correctness differs; (ii) **union repair rate** — the fraction of bare errors some population member repairs; (iii)

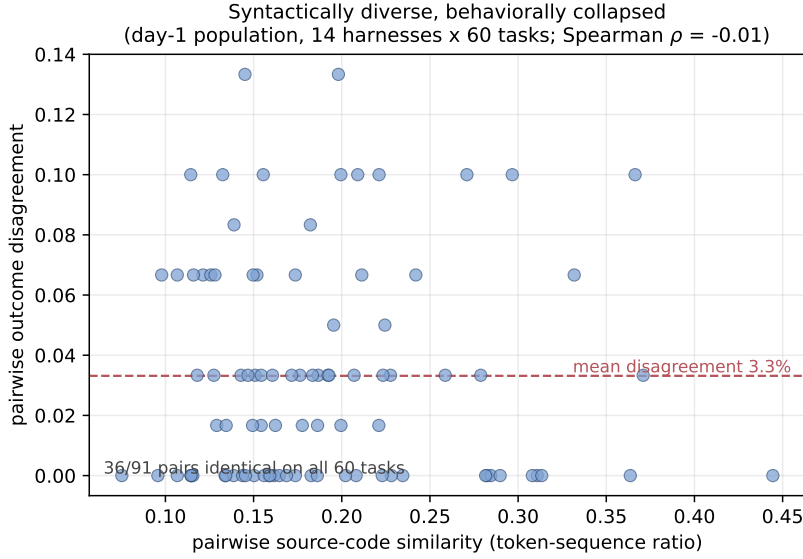


Figure 1: **Syntactic diversity vs. behavioral collapse** (Phase I, discovery; day-1 population: 14 harnesses $\times$ 60 tasks, 91 pairs including the bare and react baselines). Each point is a harness pair; x = token-sequence code similarity, y = fraction of tasks with differing correctness. Code similarity does not predict outcome disagreement ($\rho = -0.01$); 36/91 pairs are identical on all 60 tasks. Raw pairs: `fig1_phenomenon_data.json` in the code release.

bare-inclusive oracle headroom

$$H(P) = \text{Acc}(x \mapsto \max_{H \in P \cup \{H_0\}} Y(x, H)) - \max_{H \in P \cup \{H_0\}} \text{Acc}(H).$$

Positive headroom means population value is *conditional on the item* — the signal a router needs. The metrics are diagnostic in combination: high disagreement with no repair is noise; repair with no headroom means one harness dominates.

3.2 THE COLLAPSE, QUANTIFIED

The domain is BIRD text-to-SQL (SQLite; execution-accuracy judging), target frozen at **GLM-5.3-Flash** (temperature 0) throughout Phase I. On the day-1 population — 14 harnesses (bare, react, and 12 evolution candidates) $\times$ 60 tasks — average pairwise outcome disagreement is **3.3%** and **36 of 91 pairs are identical on every task**; among the 12 generated candidates alone (66 candidate pairs; mean pairwise code similarity 0.143 under the normalized-character metric of the original code-difference check), disagreement is 3.8% with 21 identical pairs. Every candidate had passed a code-difference check (“all files distinct, non-trivial diffs”). Union repair over bare errors is 7.7% and oracle headroom 3.3 pp; react differs from bare on zero tasks.

Figure 1 makes the central relation quantitative: across all 91 pairs, pairwise source-code similarity under the figure’s token-sequence metric (mean 0.190; distinct from the candidates-only normalized-character metric of 0.143 above) has Spearman $\rho = -0.010$ with outcome disagreement — code similarity shows *no observed rank association* with behavioral similarity in this population. (Pairs share harnesses and are not independent samples; we report ρ as a descriptive summary, not an inferential test.)

3.3 TRACE TAXONOMY: WHY CODE DIVERSITY FAILS TO BECOME BEHAVIORAL DIVERSITY

Auditing execution traces (the sequence of LLM calls and artifact executions per task) decomposes “different code” into three failure classes:

- **T1 — mechanism described, not executed (6/12 candidates).** The harness issues exactly one LLM call and never executes an artifact — the identical execution path as `bare`. Docstrings claim retrieval, self-checking, or token caps; none exists in the control flow.
- **T2 — feedback without a causal path (`react`, 10/10 audited tasks).** SQL is executed and errors fed back, yet the final SQL is byte-identical to `bare`’s on every audited task.
- **T3 — broken multi-turn path (1/12).** The one candidate implementing genuine multi-turn repair returns prose as final SQL, converting `bare`-correct items into harness-wrong.

The taxonomy is actionable: T1 demands an execution-path acceptance gate, T2 demands verified causal paths for feedback, T3 demands executable-output validation — all generation-protocol requirements.

3.4 HUMAN POSITIVE CONTROL: THE HEADROOM EXISTS

To exclude the alternative that the domain simply lacks instance-conditional signal, we wrote four harnesses by hand with forcibly distinct execution paths (`hpc_repair`: execute-and-repair; `hpc_vote3`: 3-vote on execution results; `hpc_schema`: schema-linking; `hpc_hint`: hint-first reordering). Same frozen target, tasks, and judge as the collapsed population: oracle headroom **6.67 pp** on 60 tasks and **7.28 pp** [3.3, 10.0] on the 151-item set, with repair/schema/hint each producing repairs the others miss. One disclosed caveat: the control was designed *after* inspecting day-1 traces, so its headroom is an existence proof, not a blind estimate. The bare baseline itself flipped 3 of 60 items between the two day-1 runs (API nondeterminism at temperature 0).

3.5 PHASE-I PROTOCOL PROBES AND THEIR LIMITS

Phase I also probed generation-side fixes — a *strategy-forced free-form* protocol (each candidate assigned an explicit mechanism, accepted after import and smoke checks) and a compiled *Harness IR*. Under the strategy-forced protocol, two of three builders produced populations whose point estimates exceeded the pre-specified diversity thresholds (Qwen 5.96, DeepSeek 7.95 pp headroom; the old-protocol arm missed at 4.64 pp), and per-strategy acceptance ordered with builder identity (GLM 3/8, Qwen 7/8, DeepSeek 8/8, under the logged re-generation tool). These observations motivate Phase II; they are *not* confirmatory, for three disclosed reasons: the design was two nested contrasts that cannot separate protocol from builder main effects; the early runs (the original Qwen/DeepSeek generations) lack per-attempt logs, so budget equality cannot be audited for them; and acceptance used an import/smoke check, not the counterfactual conformance gate defined in §4. Full condition details and provenance are in Appendix A.

Generation-null failures (M0). When we later re-ran the original optimize loop with Qwen and DeepSeek builders, its 40 emitted candidates proved byte-identical to the `bare` baseline source. Identical code trivially yields identical outcomes, so this run provides *no* evidence about behavioral collapse or builder effects; we report it as a distinct, more primitive failure mode — the pipeline produced no distinct population at all. The collapse claims above rest exclusively on the genuinely distinct-source day-1 population.

4 PHASE II: CONFIRMATORY GENERATION STUDY (DESIGN)

Phase II is the confirmatory study. Its protocol was frozen and committed before any confirmatory outcome was observed (one exception, disclosed as deviation D1 in the protocol changelog: an 18-item pipeline smoke test whose aggregate accuracies were observed and whose items were retained); deviations are logged in the protocol changelog (Appendix B). The frozen estimand, splits, instruments, and analysis plan are summarized here.

4.1 DESIGN

Arms. Two crossed factors: *strategy forcing* (S=1: the builder is assigned a named mechanism from a fixed set of eight; S=0: the builder invents an improvement with no mechanism named) and the *conformance gate* (G=1: the harness must pass the frozen conformance suite; G=0: neutral validity only). This yields four factorial arms, which we denote **II-A** (free/ungated), **II-B** (free/gated),

II-C (forced/ungated), **II-D** (forced/gated). A fifth, pre-specified *secondary* arm **II-E** (open mechanism discovery: the builder emits a self-specified behavioral contract over generic trace primitives, gated on its own declaration) tests whether mechanism diversity requires author-supplied vocabulary at all; it is deliberately outside the factorial. These arm names are Phase-II-specific; Phase-I conditions in §3.5 are always referred to with their descriptive names.

Equal raw generation budget. Every slot in every arm receives exactly $R=3$ raw builder attempts (same model, temperature 0.7, 16,384 token cap). Arms differ *only* in which raw candidate is admitted: ungated arms take the first candidate passing neutral validity (parses, imports, returns SQL on a development question — mechanism-blind); gated arms take the first additionally passing the conformance suite. Slot failure is final; populations are never topped up, so generation reliability is part of the treatment. All raw candidates are retained and generation reliability is reported separately from population quality via three rates: R_{artifact} (valid artifact), R_{contract} (valid, non-vacuous contract), R_{fidelity} (asserted properties verified, conditional on a valid contract).

Builders and seeds. Six builders from independent labs (GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, ERNIE-5.0-Thinking-Preview), three seeds each: 18 paired (builder $\times$ seed) cells. Every cell enters the primary contrast regardless of admitted population size, including $K=0$ — excluding low-yield cells would condition on a post-treatment outcome.

Data isolation. A fail-closed contamination registry scans every project artifact for observed BIRD questions. Two databases touched during development are banned wholesale; the confirmatory evaluation uses **1,169 questions from nine databases never opened during development** (the registry’s defensible claim is no project-specific adaptive use, not pretraining-time isolation; the 18-item smoke contact is the disclosed exception), with a stratified 400-item core drawn before any outcome existed. Factorial secondary analyses run on the same core items for all four arms.

Judge. Primary: the official BIRD execution scorer (set equality on raw result rows, 30 s wall-clock timeout). Secondary: the Phase-I legacy judge, measured to be uniformly conservative (on Phase-I matrices it never credits an answer the official scorer rejects).

Instruments. The conformance suite drives each harness against scripted scenarios and asserts on the recorded execution trace, never on task performance, so gating cannot leak outcome supervision. Every mechanism is probed with *paired counterfactual variants* whose correct answer differs — including a variant where the first query *succeeds*, separating conditional repair from an unconditional second call — and answers are compared by exact normalized equality. The suite was calibrated before any Phase-II generation against three hand-written positive controls (all must pass) and two known-negative impostor harnesses (both must be rejected), plus the invariant that the bare baseline must violate every conditional property; the calibration is an executable gate re-run before each generation batch. Arm II-E additionally enforces non-vacuity (the contract must be violated by the bare baseline, checked by running it) and verifies only the properties actually asserted.

4.2 FROZEN ANALYSIS PLAN

The **primary estimand** is the bare-inclusive oracle headroom $H(P)$ (Equation in §3.1; $K=0$ gives $H = 0$, and $K=1$ is a genuine bare-vs-harness routing problem) of arm II-D minus arm II-A, paired by (builder $\times$ seed), on the full 1,169-question set under the official judge — a continuous effect with two-sided 95% CI and a paired permutation test, not a pass/fail threshold. The Phase-I thresholds (union repair $\geq 15\%$, headroom ≥ 5 pp) are retained only as a descriptive operational checklist; threshold-crossing itself is statistically uncertain and is never used as a scientific verdict.

Confirmatory secondaries (Holm-corrected, tested after the primary, all four arms on the same core-400 items): gate main effect $[(B-A)+(D-C)]/2$; strategy main effect $[(C-A)+(D-B)]/2$; and the strategy $\times$ gate interaction $(D-C) - (B-A)$ — the key mechanism-level question of whether strategy differentiation requires execution fidelity to be realized.

Inference uses a hierarchical paired bootstrap (10,000 replicates): databases resampled with replacement, then tasks within each drawn database, with (builder $\times$ seed) cells resampled as paired units. K -matched sensitivity ($k=1 \dots K_{\min}$) and a population-conditional analysis ($K \geq 3$ both

Phase-II confirmatory design: database-level isolation, 2×2 factorial + open-mechanism arm, three-layer admission

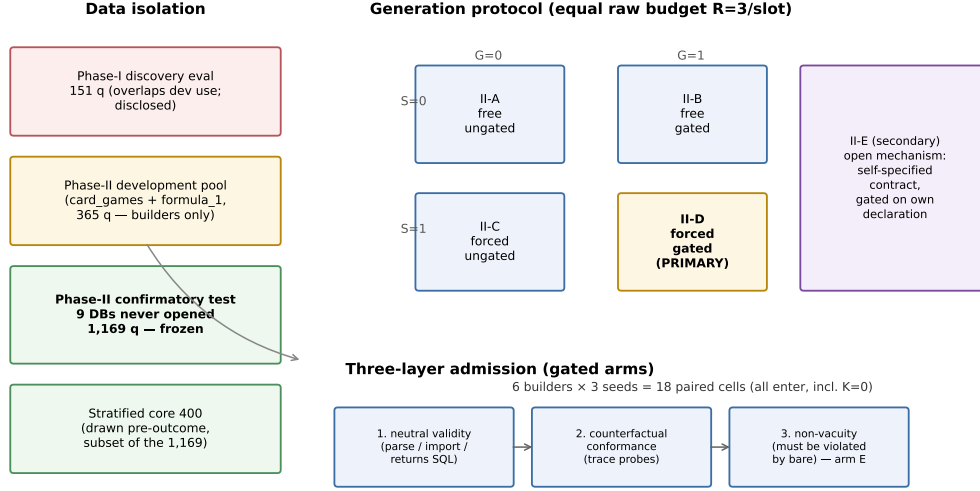


Figure 2: **Phase-II confirmatory design** (frozen protocol). Left: database-level contamination isolation (the two development databases are banned wholesale; evaluation uses nine databases never opened during development). Middle: the 2×2 factorial (strategy forcing $\times$ conformance gate) with equal raw budgets, plus the open-mechanism arm II-E. Right: the three-layer admission pipeline (neutral validity $\rightarrow$ counterfactual conformance $\rightarrow$ non-vacuity vs. bare).

	Arm II-A (free/ungated)	Arm II-D (forced/gated)
Oracle headroom $H(P)$, official judge	<i>pending</i>	<i>pending</i>
Union repair rate	<i>pending</i>	<i>pending</i>
Union harm rate	<i>pending</i>	<i>pending</i>
Pairwise disagreement	<i>pending</i>	<i>pending</i>
Unique outcome-vector fraction K_{eff}/K	<i>pending</i>	<i>pending</i>
Paired contrast $H(\text{II-D}) - H(\text{II-A})$, 18 cells:		
Point estimate	<i>pending</i>	
Two-sided 95% CI (hierarchical bootstrap)	<i>pending</i>	
Paired permutation p	<i>pending</i>	

Table 2: **Primary result (pending)**. Bare-inclusive oracle headroom per arm and the paired contrast, pooled over 6 builders $\times$ 3 seeds on the 1,169-question confirmatory set, official BIRD judge. Per-cell values, the K-matched sensitivity curve, and the population-conditional analysis will appear in Appendix B tables and the released analysis outputs.

arms) are reported as sensitivity checks. Note that $H(P)$ is *not* monotone in K : oracle accuracy is non-decreasing in K by construction, but so may be the best fixed accuracy — adding a harness that answers everything correctly drives headroom to zero — which is why matched-K comparison, not monotonicity, licenses cross-population claims.

Robustness audits (frozen): dual-judge scoring on every cell; cache-off replication of arms A and D on the core (quantifying how much apparent agreement is the shared response cache, i.e. common random numbers, versus structural); and three independent cache-off repeats of bare and arm D on the core, giving an outcome flip rate and repair stability rate.

Contrast	Effect on headroom (95% CI)
Gate main effect $[(B-A) + (D-C)]/2$	<i>pending</i>
Strategy main effect $[(C-A) + (D-B)]/2$	<i>pending</i>
Strategy $\times$ gate interaction $(D-C) - (B-A)$	<i>pending</i>

Table 3: **Confirmatory secondaries (pending)**, Holm-corrected. All four arms evaluated on the identical 400-item core.

Arm	R_{artifact}	R_{contract}	R_{fidelity}
II-A / II-C (ungated)	<i>pending</i>	—	—
II-B / II-D (gated)	<i>pending</i>	<i>pending</i>	<i>pending</i>
II-E (open mechanism)	<i>pending</i>	<i>pending</i>	<i>pending</i>

Table 4: **Generation reliability (pending)**, reported separately from population quality: a builder that cannot write a valid artifact, one that writes but over-asserts its contract, and one whose asserted behavior verifies are three different findings.

5 PHASE II: RESULTS

5.1 PRIMARY CONTRAST: FORCED+GATED VS. FREE+UNGATED

5.2 FACTORIAL DECOMPOSITION (SAME CORE-400 ITEMS)

5.3 GENERATION RELIABILITY AND ARM II-E

5.4 ROBUSTNESS AUDITS

Dual-judge agreement, cache-off replication (arms A and D), and repeat-run outcome stability (bare and arm D, three cache-off repeats) will be reported per the frozen plan. *Pending*.

6 BOUNDARY RESULT: DIVERSITY DOES NOT IMPLY LEARNABLE ROUTING

Phase I included a pre-specified value-prediction protocol on its admitted populations. Its outcome is a boundary result for the routing literature; we state its scope precisely, because the Phase-I design (task overlap, proxy judge, two populations) limits what it can establish.

Open-set value prediction does not beat the best fixed harness. A leave-one-harness-out (LOHO) predictor — logistic heads on task TF-IDF plus harness code and structural features — fails to beat the best fixed harness on both admitted Phase-I populations (DeepSeek arm -3.4 pp, Qwen arm $+0.9$ pp, bootstrap CI $[-5.2, +6.9]$; cross-protocol pool -0.9 pp, CI $[-6.0, +4.3]$). Apparent per-fold repair AUROC of 0.82–0.98 collapses under task-held-out controls: with database-stratified 50/50 task splits (five seeds, task text + harness identity features), repair classification is at chance (mean AUROC 0.512, seed range 0.35–0.60) and the induced policy matches bare ($+0.27$ pp). Within this tested configuration, the apparent signal is task memorization, not harness-value estimation. Unseen-harness representations add ≤ 0.001 AUROC over task-only features.

The one positive result is narrow. On the 24-harness cross-protocol pool only, the same predictor used as an *abstaining gate* gained $+6.9$ pp over bare with zero *observed* induced harm on its interventions. We state its scope: it is measured on the same tasks the gate was tuned on, on a pool assembled post hoc across protocols; zero observed harm on these items is not a guarantee of zero harm on deployment, and single-builder populations showed intervention harm of 2.8% and 8.7%. Selective abstention may be easier than full routing; this observation motivates, but does not establish, that claim.

We conclude: behavioral diversity is necessary for routing value to exist, and estimating an unseen harness’s per-item value from code and task text alone remains open — on the predictors, tasks, and splits tested here.

7 DISCUSSION

The precondition metrics as a community tool. The three metrics of §3.1 are computable from the outcome matrix any evaluation already produces. We argue they should be reported alongside accuracy wherever populations of generated harnesses (or agents, or scaffolds) are consumed by a selector. One caution the definition itself supplies: failing an operational threshold does not mean routing is impossible — a population with 4 pp of headroom fails a 5 pp checklist yet could still support a router that beats the best fixed option by up to 4 pp. The checklist tells you which regime you are in; it is a measurement convention, not a theorem about what a router can recover.

Implications for AI4AI generation. The taxonomy implies concrete protocol requirements: acceptance defined on *traces*, not docstrings or code diffs; feedback mechanisms with a demonstrated causal path; and executable-output validation before any capability claim. Phase II tests exactly these requirements under controlled conditions: its gate verifies mechanism execution via counterfactual probes, and its factorial design can attribute effects to the gate, to strategy forcing, and to their interaction, rather than to end-to-end system differences.

Reliability and quality are different axes. Phase-I observations suggested builder identity governs generation reliability (3/8, 7/8, 8/8 strategies accepted) more than per-strategy solution quality; Phase II makes this decomposition explicit by reporting R_{artifact} , R_{contract} , and R_{fidelity} separately from population headroom, under equal raw budgets. A protocol that reliably produces nothing and one that produces diverse-but-redundant populations are different failures, and the design can tell them apart.

Limitations. One domain family (text-to-SQL); Phase-I results come from a discovery design with disclosed task overlap and a proxy judge, and the human control was designed after inspecting development traces. Phase-II results were pending at the time this draft was assembled and will be reported under the frozen analysis plan without reference to their direction. The routing boundary result is established on the tested predictors, tasks, and splits only. Early Phase-I generations lack per-attempt logs (disclosed in Appendix A); Phase II closes this gap for all of its runs.

8 CONCLUSION

AI-generated harness populations can be exactly what their source files suggest and nothing like what routing needs. We measured that gap on a genuinely distinct-source population — code similarity shows no observed rank association with outcome disagreement — explained its mechanism at the trace level, and showed the missing headroom exists on the task. We separated genuine collapse from generation-null failures, and froze a confirmatory factorial study that isolates the generation-side factors under uncontaminated evaluation.

The Phase-II conclusion will be selected from the following pre-drafted statements according to the frozen analysis, without reference to which is most flattering:

- **If the primary contrast is positive with CI excluding zero:** explicit strategy assignment plus counterfactual conformance gating increases the routing-relevant headroom of generated harness populations under equal raw budgets; the factorial decomposition then locates the effect (gate, forcing, or interaction).
- **If the contrast shows no clear improvement (CI overlapping zero):** the data do not support a claim that the combined intervention increases population headroom; Phase-I’s positive point estimates would then be protocol- or sample-specific, but an unresolved estimate is not itself evidence of no effect.
- **If the contrast is negative with CI excluding zero:** the gated-forced protocol reduces population headroom (e.g. by concentrating quality or costing generation yield); the failure mode itself is informative for harness-generation design.

In all cases the practical output stands: measure disagreement, union repair, and headroom before routing over a generated population; gate generation on verified execution, not on text.

AI USE STATEMENT

This work is, by construction, an AI-assisted research project studying AI-generated artifacts, and we disclose the extent of that assistance precisely.

Required disclosure. We used generative AI tools for: (i) writing and modifying the experimental code (generation pipeline, evaluation collectors, statistical analysis scripts), which was executed and verified through automated tests, calibration against hand-written positive controls and known-negative mutants, and cross-model adversarial review; (ii) executing experiments, including all model API calls reported; (iii) analyzing experimental data and producing tables and figures; and (iv) drafting and editing manuscript text. Every quantitative claim in this paper is generated by committed, version-controlled analysis scripts from archived raw outputs, and every deviation from the pre-frozen protocol is disclosed in the protocol changelog.

Research ideas and direction. The core research questions, the causal design decisions (estimand, contrasts, exclusion rules), the interpretation standards, and all go/no-go decisions were made by the human authors, with AI tools acting as executor and as one adversarial reviewer (a different model family from the executor, per our cross-model review discipline).

Responsibility. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI. The factual statements in this section are provided for author verification; final responsibility for their accuracy rests with the authors.

REFERENCES

- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026.
- Yong-eun Cho. It’s not the capability: Harness sensitivity is non-monotone across LLM agent tiers. *arXiv preprint arXiv:2605.26731*, 2026.
- Tianyu Fan and Chao Huang. HELIX: Model-harness co-evolution for recursive self-improvement. *arXiv preprint arXiv:2608.13951*, 2026.
- Akshat Gupta, Jermaine Lei, Alexander Lu, Gopala Anumanchipalli, and Leshem Choshen. Automated discovery has no universally superior harness. *arXiv preprint arXiv:2607.18235*, 2026.
- Minho Kim. When should LLMs search? counterfactual supervision for search routing. *arXiv preprint arXiv:2607.05752*, 2026.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIRD’s eye view. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, and Hang Yan. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.
- Xinchen Liu, Hang Zhou, Yingjie Zong, Yuchuan Tian, Liuyang Song, and Shuo Zhang. Agentic routing: The harness-native data flywheel. *arXiv preprint arXiv:2607.11399*, 2026.
- Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution. *arXiv preprint arXiv:2607.13683*, 2026.
- Atsuyuki Miyai, Kiyoharu Aizawa, and Toshihiko Yamasaki. Task-CoEvolve: Efficient harness optimization via adaptive validation task selection. *arXiv preprint arXiv:2608.20169*, 2026.
- Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, and Lisa Adams. GRASP: Gated regression-aware skill proposer for self-improving LLM agents. *arXiv preprint arXiv:2605.29668*, 2026.

Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Xinmei Tian, Bo Han, Yike Guo, et al. TTHE: Test-time harness evolution. *arXiv preprint arXiv:2607.08124*, 2026.

Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, and Junyao Yang. Harness handbook: Making evolving agent harnesses readable, navigable, and editable. *arXiv preprint arXiv:2607.13285*, 2026.

Jinghan Xu, Yikai Zhang, Aili Chen, Weiyan Li, Jiaqing Liang, and Deqing Yang. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification. *arXiv preprint arXiv:2608.27311*, 2026.

Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, and Zhifei Xie. JIT-Agent: Scaling harness intelligence via just-in-time harness evolution. *arXiv preprint arXiv:2608.25593*, 2026a.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, and Yiqun Zhang. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026b.

Heng Zhou, Zelin Tan, Zhemeng Zhang, Yutao Fan, Yibing Lin, and Li Kang. Select-then-solve: Paradigm routing as inference-time optimization for LLM agents. *arXiv preprint arXiv:2604.06753*, 2026.

A PHASE-I PROVENANCE AND CONDITIONS (DETAIL)

Discovery status. Phase I (Sections 3 and 6) is a discovery study. 24 of its 151 evaluation items overlapped day-1 protocol-development measurements; the human control was designed after inspecting day-1 traces (its headroom is an existence proof, not a blind estimate); the repository history was initialized retrospectively, and the decision-log changelog is the authoritative ordering record. Phase-I admission thresholds (union repair $\geq 15\%$, headroom ≥ 5 pp, ≥ 3 pairwise-distinct non-empty fix sets) were frozen in the internal decision log before the 151-item evaluation; the v6 $\rightarrow$ v7 threshold change (20% $\rightarrow$ 15%) is disclosed, and both readings admit the same populations. This is an internal pre-evaluation freeze, not external preregistration.

Data hygiene. The bare baseline flipped 3 of 60 items between the two day-1 runs (API nondeterminism at temperature 0), so 60-item columns are conditioned on slightly different bare-error sets. Phase-I judging used a set-of-rows-equality proxy that we later measured to be uniformly conservative relative to the official BIRD scorer (on the two Phase-I matrices with logged final SQL, it never credits an answer the official scorer rejects; agreement 0.97–0.99).

Phase-I populations. Seed baselines: `bare` (single call) and `react` (execution feedback loop, shipped with TTHE). *Old proposer protocol*: six candidates selected from TTHE’s optimize-loop output (GLM builder), selected for behavioral diversity using only the 36 questions disjoint from the evaluation set, by a rule frozen before evaluation. *Strategy-forced free-form*: GLM, Qwen, and DeepSeek builders, eight assigned mechanisms each; the GLM population is the logged re-run (K=3; the original 09-01 run recorded zero accepted outputs but its tool was not preserved and the observation is unaudited). *Harness IR*: six harnesses compiled deterministically from declarative mechanism specifications (context $\in \{\text{plain, schema.link, hint.guard, link+hint}\}$, candidate generation with temperature, per-candidate execution, repair rounds, selection rule), so a declared mechanism is present in the control flow by construction. *Human positive control*: the four hand-written harnesses of §3.4.

Generation-provenance gaps. The 09-01 free-form populations were produced in-session; their generation tool was not preserved, and per-attempt logs do not exist for them — so budget equality cannot be audited for those runs, and we do not claim it. Their acceptance was an import/smoke check at evaluation time, *not* the Phase-II counterfactual conformance gate. The tool of record from 2026-09-03 (`builder_generate3.py`) logs every attempt (3 retries, 16,384-token budget, per-attempt stage and finish reason); the completed per-strategy grids and the builder-acceptance ordering (3/8, 7/8, 8/8) come from this logged tool and are post-hoc with respect to the frozen admission decisions.

B PHASE-II PROTOCOL SUMMARY AND DEVIATION LOG

The authoritative protocol (estimands, splits, instruments, exclusion rules, and the complete deviation changelog D1–D13) is `PHASE2_PROTOCOL.md` in the code release, frozen at the commit recorded in `PROTOCOL_FREEZE.txt`. Summary: 2×2 factorial plus open-mechanism arm; six builders $\times$ three seeds; equal raw budget $R=3$ per slot with no top-up; contamination registry (fail-closed, two development databases banned wholesale); 1,169-question confirmatory set from nine databases never opened during development plus a pre-drawn 400-item core; official BIRD judge with enforced 30 s timeout; response cache keyed on model identity with cross-process locking (deviation D13 documents and fixes a cache concurrency bug found during collection); primary contrast D–A on paired (builder $\times$ seed) cells with hierarchical bootstrap and permutation inference; Holm-corrected factorial secondaries on the common core; K-matched sensitivity only. Deviations D1–D13 include the disclosed 18-item pipeline smoke test, instrument-calibration fixes made before any confirmatory outcome was observed, the credential-handling incident and its remediation, and the artifact-divergence quarantine — each with its rationale and date in the changelog.

C PHASE-I ADDITIONAL RESULTS

Per-harness accuracies, per-fold LOHO tables, bootstrap intervals, fix-set audits, K-subsampling curves, the task-held-out control, and the second-target replication (Qwen3.8-Flash as frozen target: collapse replicates; the strategy-forced population retains headroom; the prompt-side human control does not transfer wholesale) are included in the anonymous code release with their generating scripts.